

SYSTEM DESIGN INTERVIEW

Design a Fraud Detection System

real-time risk scoring for cards, ACH, and account-to-account transfers at issuer scale

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01 · Problem Statement
- 02 · Requirements
- 03 · Capacity Estimation
- 04 · High-Level Architecture
- 05 · Layered Defence
- 06 · Feature Store
- 07 · Online Feature Aggregation
- 08 · Model Serving
- 09 · Decision Flow
- 10 · Feedback Loop

- [11](#) · [Storage Plan](#)
- [12](#) · [Comparisons](#)
- [13](#) · [Failure Modes & Recovery](#)
- [14](#) · [Design Trade-offs](#)
- [15](#) · [Interview Playbook](#)

01

Problem Statement

Real-time fraud scoring on the auth path

Design a real-time fraud detection system for a global card issuer / payments platform. Every card swipe, ACH origination, and account-to-account transfer must be scored **in-line on the authorization path** in under 50 ms p99, with a verdict of ALLOW, REVIEW, or DECLINE. The system blends a deterministic rules engine (velocity caps, blocked geographies, BIN blocklists) with a gradient-boosted ML model fed by a streaming feature store, and feeds confirmed-fraud labels back into a nightly retraining loop.

The companion *Payment Gateway* guide covers the auth/capture/ledger path end to end; this document zooms into the **risk service, feature store, model server, and feedback loop** that the gateway calls during the 'fraud check' step. Treat the gateway as a fixed upstream caller — our SLO is everything between POST /score and the verdict response.

Clarifying Questions

Question	Assumption
Transaction types?	Cards (issuer side), ACH originations, real-time A2A transfers
Scoring mode?	Synchronous in-line on auth (must respond before the network does)
Inline latency?	p99 < 50 ms end-to-end including feature lookup + model inference
Throughput?	~12K TPS sustained, ~50K TPS Black Friday peak
Decisions?	ALLOW (auto-approve) / REVIEW (queue) / DECLINE (block)
Fraud rate?	~30 bps (0.3%) of transactions are confirmed fraud
Labels?	Chargebacks (T+30–120 d) + customer fraud reports + manual-review outcomes
Explainability?	Required for declines (regulator + dispute evidence)
Retraining?	Nightly offline; champion/challenger A/B for promotion

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Score transaction	POST /score → returns {decision, score, reason_codes} on the auth path
Rules pre-filter	Hard-block known-bad (sanctioned countries, BIN blocklist, velocity caps)
ML scoring	Gradient-boosted model on aggregated features; score 0–1000
Manual review queue	Mid-band scores route to analyst console with case context
Feedback ingestion	Chargebacks, customer reports, analyst dispositions become labels
Nightly retraining	Train on last 90 days; promote via shadow + champion/challenger
Audit trail	Every decision (score, features, model version, rule hits) persisted 7 yr
Explainability	SHAP-style top-3 reason codes returned with every decision

Non-Functional Requirements

Requirement	Target
Inline latency	p99 < 50 ms (feature lookup + inference); p50 < 12 ms
Availability	99.99% on the scoring path; degrade to rules-only on model failure
Throughput	~12K TPS sustained, ~50K TPS peak; design headroom 2× peak
Freshness	Streaming features visible < 1 s after the upstream txn
Retraining	Nightly offline; full retrain in < 4 h on the warehouse
Drift detection	PSI/KS on features daily; AUC-ROC on labels weekly
Audit retention	7 years (PCI / SOX / chargeback dispute window)

03

Capacity Estimation

Math for issuer scale

Traffic

- Daily transactions: **~1B** (issuer-scale; cards + ACH + A2A combined)
- Sustained TPS: $1B / 86,400 \approx \text{~12K TPS}$ daily average; ~25K TPS at peak hour
- Black Friday / Cyber Monday peak: bursts to **~50K TPS**; design headroom to 2× peak (~100K TPS) so we never shed load on the auth path
- Read amplification per scored txn: ~30 feature lookups (multi-window aggregates per user/card/device/merchant) → **~360K Redis GETs/sec sustained, ~1.5M/sec peak**
- Confirmed-fraud rate: ~30 bps → **~3M fraud events/day**, the labelled tail of the funnel

Latency Budget (50 ms p99 inline)

Stage	Budget (p99)	Notes
Network in (gateway → risk)	3 ms	Same-AZ gRPC; mTLS; pooled connections
Rules engine pre-filter	5 ms	In-process; deterministic; short-circuits on hard-block
Feature lookup (Redis)	12 ms	~30 keys; pipelined MGET; hot store in same AZ
Model inference	20 ms	LightGBM / DNN; co-located server; warm process
Score → decision	2 ms	Threshold lookup + reason codes
Audit log write (async)	0 ms inline	Fire-and-forget to Kafka; not on critical path
Network out	3 ms	Reply gRPC
Headroom	5 ms	Sums to 50 ms with slack for tail spikes

Storage

- **Transaction events (Kafka):** $1B/day \times \sim 1.5 \text{ KB} \approx \text{1.5 TB/day raw}$; 7-day retention on hot tier $\approx \sim 10 \text{ TB}$; replicated 3× $\approx \sim 30 \text{ TB}$ cluster
- **Feature store hot (Redis):** $\sim 500M \text{ active entities} \times \sim 200 \text{ features} \times \sim 16 \text{ B} \approx \text{~1.6 TB hot working set}$; cluster sized at 2.5 TB to leave headroom
- **Feature store warm (Cassandra / Bigtable):** 12 months of windowed aggregates $\approx \sim 120 \text{ TB}$ across all entities and windows

- **Decision audit log (Postgres + S3):** $1\text{B/day} \times \sim 2\text{ KB} \approx \sim 2\text{ TB/day}$; 90 days online ($\sim 180\text{ TB}$), 7 yr in object storage ($\sim 5\text{ PB}$ Glacier)
- **Training warehouse:** last 90 days of features + labels $\approx \sim 10\text{ TB}$ in Snowflake / BigQuery

KEY

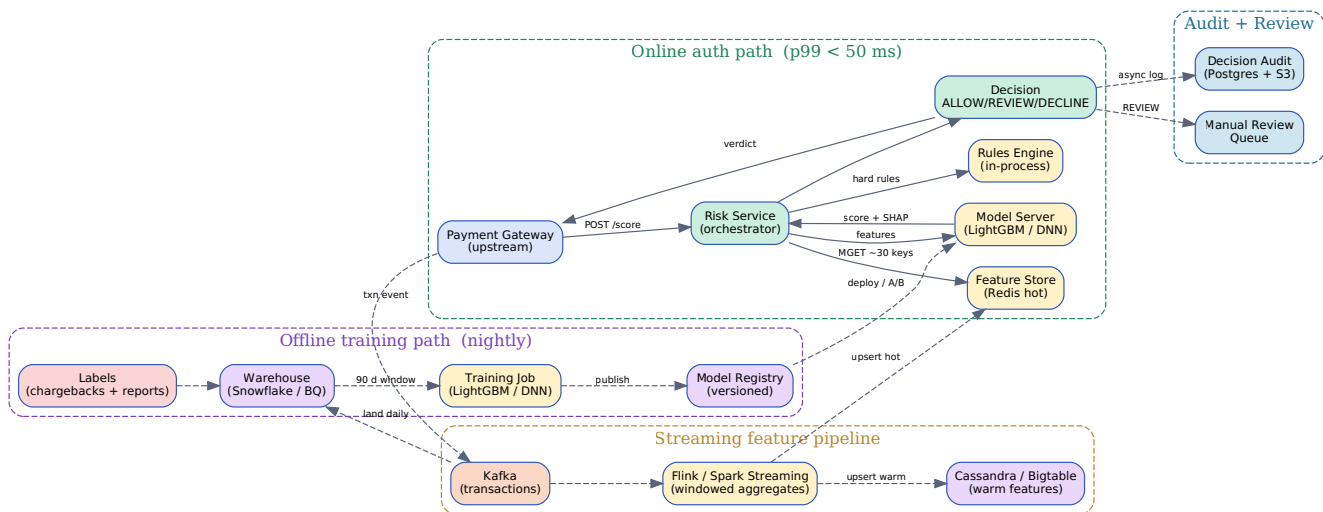
Key Numbers

1B txns/day · $\sim 12\text{K TPS}$ sustained · $\sim 50\text{K TPS}$ Black Friday peak · p99 inline $< 50\text{ ms}$ · ~ 30 feature lookups per scored txn · $\sim 1.6\text{ TB}$ hot Redis feature store · $\sim 30\text{ bps}$ fraud rate · nightly retrain on 90 d window.

High-Level Architecture

Online auth path + offline training

The system has two clearly separated planes: an **online scoring path** (gateway → risk service → feature store + rules + model server → decision) bound by a 50 ms budget, and an **offline training path** (transaction events → warehouse → daily training → model registry) that updates the model the online path serves. A streaming feature pipeline (Flink) bridges the two: it consumes the same transaction events that flow through the online path and continually updates the feature store so that the next inference sees the very latest aggregates.



Online scoring path (solid) and offline training path (dashed). The risk service calls feature store + rules engine + model server in parallel where possible; transactions also fan out to Kafka for streaming feature updates and warehouse landing for nightly retraining.

Component Roles

- **Risk Service:** stateless orchestrator that owns the 50 ms budget; fans out to feature store + rules + model server, combines into a verdict
- **Rules Engine:** in-process deterministic short-circuit (hard-block sanctioned countries, BIN blocklist, velocity caps); explainable and auditable
- **Feature Store:** Redis hot (last 24 h aggregates) + Cassandra warm (30 d / 12 mo aggregates); served by feature retrieval API
- **Model Server:** co-located process (Triton / TF-Serving / custom); warm model in memory; supports shadow + champion/challenger A/B
- **Streaming pipeline:** Flink job consumes the txn event stream and continually updates feature store; bounded delay < 1 s

- **Offline trainer:** nightly Spark / Ray job pulls 90 d of features + labels from the warehouse, trains, evaluates, and publishes to the model registry
- **Manual review queue:** mid-band scores routed here; analyst dispositions feed back as labels
- **Decision audit log:** every verdict (score, features used, model version, rule hits) → Postgres (90 d hot) + S3 (7 yr cold) for chargeback dispute evidence

05

Layered Defence

Rules + ML + manual review

Fraud is an adversarial game: rules are explainable but bleed against novel patterns; ML catches novel patterns but is opaque to regulators and risk teams. Mature systems combine all three layers, each responsible for a different type of error.

Layer 1: Rules Engine (Pre-Filter)

- **Velocity caps:** same card > 5 auths in 60 s → hard decline; same device > 10 cards in 24 h → review
- **Geo / sanctions:** issuing country in OFAC SDN list → hard decline; impossible-travel (US auth then RU auth 30 min later) → review
- **BIN blacklist:** known-compromised BINs from card-network bulletins → hard decline
- **Allow / deny lists:** per-merchant overrides for known good (whitelist) and known bad (blacklist) cards / customers
- **Properties:** deterministic, explainable, ~1–2 ms in-process, ships hourly via config push

Layer 2: ML Model

- **Algorithm:** LightGBM / XGBoost (gradient-boosted trees) for tabular features; optional DNN for sequence / embedding features
- **Inputs:** ~200 features pulled from the feature store at multiple windows (1m / 1h / 1d / 30d) per (user, card, device, merchant)
- **Output:** calibrated risk score 0–1000; SHAP top-3 reason codes returned with the score
- **Latency:** p99 < 20 ms in-process; warm model; no GPU on the auth path
- **Class imbalance:** ~30 bps fraud → undersample non-fraud, oversample fraud, or use focal loss; track precision/recall, not raw accuracy

Layer 3: Manual Review Queue

- Mid-band scores (e.g. 600–800) route to a human analyst console with full case context: txn details, recent activity, device fingerprint, geo, model reason codes
- Analyst SLA: typical 2–5 min for high-value txns; auto-approve with timeout for low-value reviews to keep the merchant happy
- Analyst disposition (CONFIRMED_FRAUD / LEGITIMATE / UNCLEAR) feeds back as a high-quality label for retraining

INSIGHT

Why All Three

Rules carry the regulatory and explainability load (you can show a regulator a flowchart). The ML model carries the adaptability load (catches patterns no analyst has written a rule for yet). The review queue carries the high-value-edge-case load (a human eyeballs the \$50K wire transfer that scored 0.72). Removing any one layer makes the other two demonstrably worse.

06

Feature Store

Multi-window aggregates per entity

The feature store is the single most important system in fraud detection — fraud signal lives in *aggregates over time*, not in the raw txn. We maintain pre-computed counters and sums per **(entity, window)** across four entity classes and four window sizes.

Entities × Windows

Entity	Examples of features	Why it matters
User	Auths/min, \$/h, distinct merchants/d, distinct countries/30d	Account-takeover patterns; ATO bursts within minutes of credential theft
Card / PAN	Failed CVV/h, declines/d, MCC entropy/30d	Card-testing rings cycle through stolen PANs at high velocity
Device	Cards used/d, IPs/d, accounts touched/30d	One device hammering many cards is a near-certain sign of fraud
Merchant	Chargeback rate/30d, fraud_score histogram, MCC	Merchant base rate: a high-CB merchant adjusts the prior on every txn

Window Sizes

- **1 minute:** burst detection (card-testing, credential-stuffing)
- **1 hour:** session-level patterns (legit checkout vs. spree)
- **1 day:** daily limits, normal customer behaviour
- **30 days:** long-term baseline, tenure, chargeback history

Hot vs Warm Storage

Tier	Store	Holds	Latency	Why
Hot	Redis (cluster)	1 m / 1 h / 1 d aggregates	< 1 ms / key	Auth path; 30 keys per scored txn; pipelined MGET
Warm	Cassandra / Bigtable	30 d / 12 mo aggregates	5–10 ms / key	Less hot; tolerable for scoring; cheaper at petabyte
Cold	S3 (parquet)			Warehouse landing for nightly training

Tier	Store	Holds	Latency	Why
		Raw events for retraining	Minutes (batch)	

Feature Aggregate Row (Redis hot)

```
# Redis key layout (one key per (entity, window))
Key:   feat:{entity_type}:{entity_id}:{window}
      e.g. feat:user:u_92f1:1m
Value: hash {
    n           : 17,           # txn count in window
    sum_amount_cts : 4_823_00,   # sum of cents
    distinct_mcc  : 4,           # distinct merchant categories
    distinct_geo  : 2,           # distinct countries
    n_decline     : 1,           # declines in window
    last_ts       : 1746537812,  # epoch sec; for window slide
    fraud_label_30d: 0           # any confirmed fraud in last 30 d?
}
TTL:   2 × window length (so a 1m window persists 2 m as a safety margin)

# At score time, the risk service builds the feature vector by
# pipelining MGET across ~30 keys (4 entities × multiple windows + scalars).
```

TIP

Train/Serve Skew is the Killer

The single most painful bug in production ML for fraud is *train/serve skew*: features computed differently offline (in Spark) vs online (in Flink/Redis). Mitigate with a single feature definition library (e.g. Feast) that emits both the streaming and batch implementations from the same source, plus daily PSI checks comparing online and offline distributions.

07

Online Feature Aggregation

Streaming windowed counters

The streaming pipeline is what keeps Redis fresh. A Flink job consumes the transactions Kafka topic, partitioned by entity ID, and continuously updates per-(entity, window) aggregates. The hot path of this update needs to be lock-free, so we use a sliding-window counter pattern and atomic Redis HINCRBY.

Sliding-Window Counter

```
# Flink operator pseudo-code: per-(entity, window) sliding aggregate.
# Each Kafka record represents one transaction; we update counters
# for *every* window the txn belongs to (1m, 1h, 1d).

def on_transaction(txn, redis, now_ts):
    for entity_type, entity_id in (
        ('user', txn.user_id),
        ('card', txn.card_token),
        ('device', txn.device_fp),
        ('merchant', txn.merchant_id),
    ):
        for window, length in (('1m', 60), ('1h', 3600), ('1d', 86400)):
            key = f'feat:{entity_type}:{entity_id}:{window}'
            # HINCRBY is atomic; pipelined for batch efficiency.
            pipe = redis.pipeline()
            pipe.hincrby(key, 'n', 1)
            pipe.hincrby(key, 'sum_amount_cts', txn.amount_cts)
            if txn.status == 'DECLINED':
                pipe.hincrby(key, 'n_decline', 1)
            pipe.hset(key, 'last_ts', now_ts)
            pipe.expire(key, 2 * length) # auto-evict stale windows
            pipe.execute()

# Distinct counts (distinct_mcc, distinct_geo) use HyperLogLog
# (PFADD) to keep the per-key memory bounded at ~12 KB regardless of
# cardinality, with ~1% error -- accuracy we can live with for fraud.

# Champion/challenger traffic split for safe model rollouts:
def route(txn, hash_seed='fraud_v1'):
    bucket = mmh3.hash(f'{hash_seed}:{txn.id}') % 100
    return 'challenger' if bucket < 5 else 'champion' # 5% to challenger
```

Why TTL = 2 × window

- **1× window:** race-prone — a window can expire mid-update and lose the increment that just fired

- **2× window:** safety margin; the next read will see the freshly slid value
- **Trade-off:** doubles Redis memory for short windows, but short windows are the cheapest to store anyway

Late / Out-of-Order Events

- **Watermarking:** Flink emits per-key watermarks; events older than 30 s are routed to a side output and reconciled offline (rare)
- **Idempotency:** txn_id deduped at the source; same Kafka offset is processed at-most-once per Flink checkpoint
- **Replay:** on Flink restart, replay from the last checkpoint; aggregates are deterministic given the input stream

08

Model Serving

Triton / TF-Serving on the auth path

Co-Located Model Server

- **Pattern:** model server runs as a sidecar / in-process library next to the risk service to avoid an extra network hop on the auth path
- **Engines:** NVIDIA Triton (multi-framework, ONNX), TF-Serving (TF / Keras), or a thin custom runner over LightGBM's C API for the fastest path
- **Warmup:** on deploy, run 1,000 synthetic requests through the model before taking traffic — first-request JIT cost is real and shows up on tail
- **No GPU:** tabular gradient-boosted trees are CPU-bound and dominated by feature lookup; GPU adds variance without throughput

Champion / Challenger A/B

- **Champion:** the model currently in production; takes 95% of traffic by default
- **Challenger:** a candidate model trained on newer data or a new architecture; takes 5% to start
- **Split:** deterministic by hash(txn_id) so the same txn always lands in the same bucket — required for clean comparison
- **Promotion:** nightly evaluation on offline-relabelled traffic; promote when challenger AUC $\geq$ champion AUC at fixed FP rate
- **Kill switch:** single config flip reverts to last-good model in < 60 s; dashboards on score distribution PSI catch silent regressions

Shadow Mode

- Net-new models go to **shadow mode** first: they score every transaction but their verdict is logged, not used
- Lets you compare distribution, latency, and downstream impact before any user sees the new model's decision
- Typical bake time: 7–14 days of full-traffic shadow before any A/B starts taking real decisions

Model Registry

- **Versioning:** every trained model gets `model_v{semver}+{git_sha}+{train_date}`; the registry tracks training data range, feature schema, eval metrics
- **Provenance:** a decision audit row stores the model version that produced it, so when a chargeback comes in 90 days later we can re-explain that exact decision

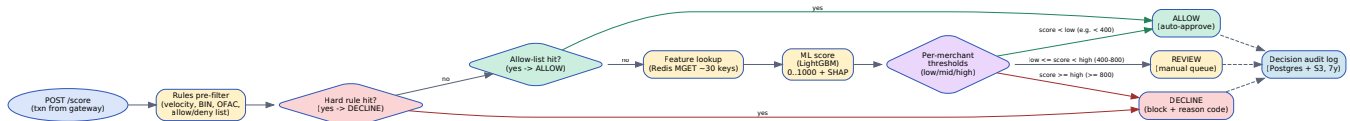
- **Rollback:** deploys are atomic flag flips backed by the registry — no copying weights around

09

Decision Flow

Score + rules → ALLOW / REVIEW / DECLINE

The verdict combines (1) a hard rules pre-filter that can short-circuit to DECLINE, (2) a model score, (3) a per-merchant + per-customer-tier threshold, and (4) post-rules that escalate edge cases to manual review. The thresholds are *tunable per merchant and per customer tier* — a luxury merchant on a premium customer might tolerate more false positives than a low-margin merchant on a new customer.



Decision flow: rules pre-filter short-circuits known-bad; ML score buckets the rest by per-merchant threshold; mid-band cases escalate to manual review.

Threshold Tuning

- **Per merchant:** a high-CB merchant gets a lower DECLINE threshold (more aggressive); a luxury merchant on premium customers gets a higher one
- **Per customer tier:** long-tenured customers with no fraud history get a generous threshold; new accounts get a tighter one
- **Per channel:** card-not-present is more aggressive than card-present; A2A transfers stricter than card swipes (because reversal is harder)
- **Per amount band:** \$50,000 wire transfers always go to manual review regardless of score

Decision Audit Log

```

CREATE TABLE fraud_decisions (
  decision_id      UUID PRIMARY KEY,
  txn_id           UUID NOT NULL,
  merchant_id      UUID NOT NULL,
  card_token       VARCHAR(255),
  user_id          UUID,
  amount_cents     BIGINT NOT NULL,
  currency         CHAR(3),
  decision         decision_enum,      -- ALLOW|REVIEW|DECLINE
  score            DECIMAL(6,2),       -- 0-1000
  model_version     VARCHAR(64) NOT NULL,
  rule_hits        TEXT[],             -- e.g. {'velocity_card_1m', 'bin_blocklist'}
  reason_codes     TEXT[],             -- top-3 SHAP features
  features_snapshot JSONB,             -- the ~200-feature vector at decision time
  threshold_low    DECIMAL(6,2),
  threshold_high   DECIMAL(6,2),

```

```
    latency_ms      INT,  
    created_at      TIMESTAMPTZ DEFAULT NOW(),  
    INDEX (txn_id),  
    INDEX (card_token, created_at),  
    INDEX (merchant_id, created_at)  
);  
  
-- Hot in Postgres for 90 d (chargeback dispute window).  
-- Archived to S3 (parquet) for the remaining 7 yr.  
-- Append-only: if a decision is later overturned, write a new  
-- correction row; never UPDATE in place. Audit trail is sacred.
```

10

Feedback Loop

Labels, retraining, drift

Fraud is a moving target: rings retool weekly, merchants change their mix, customer behaviour shifts seasonally. A model frozen for a month is already losing to the adversary. The feedback loop is what keeps the model honest.

Label Sources

Source	Latency	Quality
Chargeback files (issuer)	T+30 to T+120 d	High; cardholder-disputed; reason-coded
Customer fraud reports	Hours to days	Medium; some self-reported false alarms
Manual review dispositions	Minutes	High; analyst-judged; explicit label
3DS challenge outcomes	Seconds	Low; only proxies fraud risk, not actual fraud
Network fraud bulletins	Days	Medium; broad-stroke (e.g. 'BIN compromise')

Nightly Retraining

1. Land all txn events of the day into the warehouse (S3 / Snowflake / BigQuery)
2. Join with labels: chargebacks come in late, so today's training set sees labels for transactions up to ~30 days ago
3. Recompute training features *using the same definitions* as the streaming pipeline (single source of truth library)
4. Train champion-arch model on last 90 d; train challenger arch in parallel
5. Evaluate offline: AUC-ROC, precision @ fixed recall, KS statistic; check for regression vs current champion
6. Publish to model registry with full metadata; **do not auto-promote** — promotion is gated on shadow + A/B

Concept Drift

- **Feature drift:** input distribution moves (e.g. mobile share rises, IP-geo skew shifts); detect with daily PSI / KS on each feature
- **Label drift:** base rate of fraud changes; check daily fraud rate vs trailing 30 d

- **Concept drift:** the relationship between features and fraud changes (e.g. fraud rings adopt a new technique); detect with rolling AUC on labelled traffic
- **Mitigation:** shadow models always running on current traffic; champion/challenger keeps a fresh candidate ready; emergency hot-fix path is a config flip to revert to last-good

WARNING

Survivor Bias

Your training set only contains decisions you *didn't block* — you can never know what would have happened on the txns you declined. Mitigations: (1) reserve 1–2% of high-score traffic to *let through* as exploration so you can observe the outcome; (2) use the manual review queue to generate labels on borderline txns. Failing to do this means your model gets progressively more confident on its own decisions and progressively blinder to its own errors.

11

Storage Plan

Where everything lives

Data	Store	Retention	Why
Transaction event stream	Kafka	7 d hot, 30 d compacted	Source of truth for the streaming pipeline; replayable on Flink restart
Hot features (1m / 1h / 1d aggregates)	Redis cluster	TTL = 2 × window	Sub-ms reads on the auth path
Warm features (30 d / 12 mo aggregates)	Cassandra / Bigtable	12 months	Cheaper at petabyte; 5–10 ms reads acceptable for the long-tail features
Decision audit log (hot)	Postgres	90 d	Chargeback dispute window; queryable from the analyst console
Decision audit log (cold)	S3 (parquet)	7 yr	PCI / SOX / regulator + dispute evidence beyond 90 d
Training data + labels	Snowflake / BigQuery	90 d active, 2 yr archive	Nightly retrain + ad-hoc analysis
Model registry	S3 + Postgres metadata	Forever	Provenance: every decision row references a model version that must exist forever
Manual review queue	Postgres	30 d active	Open cases live here; closed cases flow to the audit log

TIP

Why Postgres for Audit

The decision audit log is mostly written and rarely read — but when it is read it is during a chargeback dispute and the answer had better be correct. Postgres gives us strong consistency on the write path (no ‘eventually durable’ surprises), familiar indexes for the analyst console, and easy export to S3 for the cold tail. Cassandra for audit would tempt us into eventual consistency and trade away the one property we cannot trade.

12

Comparisons

Choices and trade-offs

Rules vs ML vs Hybrid

Approach	Strengths	Weaknesses
Rules only	Explainable, fast, regulator-friendly, easy to ship hotfix	Bleeds against novel fraud; brittle; rule explosion (10K+ rules unmaintainable)
ML only	Catches novel patterns; adapts; high recall for the precision	Opaque to regulators; hard to hotfix; vulnerable to concept drift; survivor bias
Hybrid (rules + ML)	Rules carry the regulatory and known-bad load; ML carries adaptability; both layers explainable	Operational complexity; two systems to maintain; need shared feature definitions

Gradient Boosting vs DNN vs Graph Models

Family	Best at	Cost	Verdict
LightGBM / XGBoost (GBT)	Tabular features; quick to train; CPU-fast inference (< 5 ms)	Low — single CPU; trains nightly on a beefy box	Default choice for tabular fraud; what most issuers actually run
DNN (tabular)	Sequence / embedding features; higher capacity	Higher — GPU train, careful inference; harder to debug	Use as a complement (e.g. on session sequences) feeding into the GBT
Graph neural nets	Ring detection (mule networks, device sharing)	Highest — graph compute infra; offline only	Run offline as a feature pipeline (graph embedding becomes a feature for the inline GBT)

Sync Inline vs Async Post-Auth

Mode	When	Trade-off
Sync inline (this design)	Card auth, A2A transfer, anything with a real-time decision point	Hard 50 ms budget; can DECLINE in real time; richer customer experience for legit txns; cost: tail-latency engineering is brutal

Mode	When	Trade-off
Async post-auth	ACH originations with a 1–3 day clearing window; offline batch reviews	No latency budget; can use richer (slower) features and ensemble models; cost: any decline is a reversal not a block — money has already moved
Two-tier (sync fast + async deep)	Best of both: a cheap inline model gates the auth, a deeper async model re-scores within minutes	Most operationally complex; inconsistency window (allowed inline, declined async) requires reversal flow

13

Failure Modes & Recovery

What breaks and how we survive it

Failure	Cause	Impact	Mitigation
Model server timeout	Inference > 30 ms; warmup not done; bad release	Risk service can't get a score in budget	Hard timeout; fall back to rules-only ; emit metric; auto-revert deploy after N timeouts
Feature store down (Redis)	Cluster failover, AZ outage	No fresh aggregates; ML score is uninformed	Degrade to per-transaction-only features (no history); rules engine still runs; tighten thresholds; alert on-call
Streaming pipeline (Flink) lag	Kafka consumer lag; bad job	Features stale by minutes; ML score uses old data	Monitor lag; pause challenger promotion when lag > 60 s; auto-scale Flink; stale-feature score gets a confidence penalty
Concept drift (silent)	Fraud ring changes tactics; model unaware	Recall drops without warning; chargebacks rise weeks later	PSI/KS daily on features; rolling AUC weekly on labels; shadow model always running; champion/challenger keeps fresh candidate
Bad model deployed	Training bug; data leak; wrong feature schema	Score distribution shifts; FP or FN spikes	Shadow + A/B catches before full rollout; PSI on score distribution as canary; one-flag kill-switch revert in < 60 s
Label leakage	Future info leaked into features	Offline AUC looks great; production AUC collapses	Train/serve skew tests; out-of-time validation; shadow comparison vs current champion
Manual review backlog	Volume spike; analyst staff out	REVIEW txns time out; merchants angry	SLA-based auto-approve for low-amount REVIEW after N min; auto-decline for high-amount after escalation
Audit log write fails	Postgres outage	Decision happened but no row; chargeback evidence gap	Async write to Kafka first (fire-and-forget); Postgres consumer is replayable; never block the auth path on audit

WARNING

Graceful Degradation Order

Under partial outage, drop in this order: (1) shadow model logging, (2) feature freshness (serve last-known features beyond TTL), (3) ML score (degrade to rules-only with tighter thresholds), (4) *never* the rules

engine. The auth path must always return a deterministic verdict in < 50 ms; rules-only is acceptable, no verdict is not.

14

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
False-positive vs missed-fraud cost	Per-merchant tunable thresholds; default biased toward letting borderline cases through to manual review	Each FP is a lost legitimate sale (~\$80 average ticket); each FN is a chargeback (~\$120 + ~\$25 fee + reputation). Aggressive thresholds optimise short-term loss but burn customers and merchants. We let merchants tune their own ALLOW/DECLINE bands and absorb the calibration in REVIEW.
Latency vs feature richness	30 features inline (1m/1h/1d aggregates), ~170 more available offline for re-scoring	Every feature added to the inline path is one more Redis GET in the 50 ms budget. We pre-materialise multi-window aggregates so each one is a single HMGET, not a stream computation; richer features (graph embeddings, sequence models) live in async post-auth re-scoring.
Explainability vs accuracy	LightGBM with SHAP top-3 reason codes (good explainability, near-DNN accuracy on tabular)	DNNs squeeze ~1–3% more AUC on rich features but are opaque; regulators require declined-customer explanations; SHAP on GBT is the practical sweet spot. Reserve DNN for offline ensembles whose output is a feature for the inline GBT.
Model freshness vs stability	Nightly retrain → shadow → 5% A/B → promote; never auto-promote without human gate	Daily retraining catches drift but every promotion is a risk of regression. The shadow + A/B + human-gated registry promotion gives drift-responsiveness without hot-deploying a regression to 100% of traffic.
Sync inline vs async post-auth	Inline for cards / A2A (real-time decision); async post-auth re-scoring for ACH and reversible flows	Inline catches fraud before money moves but locks us into a 50 ms budget. Async has unlimited budget but can only reverse, not block — we use both and pick by transaction type.
Rules engine vs ML model	Hybrid: rules carry hard-blocks and regulatory load, ML carries adaptive scoring	Pure rules bleed against new patterns; pure ML is opaque and brittle on hotfix. The combination is operationally heavier (two systems) but every layer is explainable, hot-fixable, and reviewable.
Strong vs eventual consistency for features	Eventual on the feature store (Redis lag < 1 s under normal load)	Strong consistency on every feature update would gate the entire txn stream behind a single writer. Eventual is fine because every aggregate is approximate by definition (a windowed counter is always 'at-least-but-maybe-1-late').

Decision	Choice	Trade-off
Audit log durability	Append-only Postgres (90 d hot) + S3 parquet (7 yr cold)	Mutable audit would let a buggy migration corrupt evidence; append-only with a model_version reference means every old decision is replayable. Cost is storage growth — accepted for the regulatory guarantee.

KEY

Headline Tension

Fraud detection has two competing obsessions: **recall** (don't miss fraud — every miss is a chargeback and a regulator note) and **precision** (don't block legit customers — every false positive is a lost sale and a furious merchant). Almost every design choice tunes the operating point on this curve, and the right operating point is per-merchant, per-customer-tier, and per-channel.

15

Interview Playbook

How to present this

This guide distils a 45-minute fraud-detection interview into a structured narrative. The trick is to keep the online and offline planes separate in your head — interviewers test whether you mix them up, because confused candidates always do.

Timeline: 5 + 5 + 10 + 20 + 5 minutes

1. **Opening (5 min):** clarify scope — sync inline scoring on auth path; cards + ACH + A2A; ~12K TPS sustained, ~50K Black Friday peak; p99 < 50 ms
2. **Capacity (5 min):** 1B txns/day, ~30 feature lookups per scored txn, ~1.6 TB hot Redis, nightly retrain on 90 d window
3. **Architecture (10 min):** Diagram 1 — split online (solid) and offline (dashed) planes; explain risk service as the orchestrator that owns the 50 ms budget
4. **Deep dives (20 min):** pick 2–3: feature store (multi-window aggregates), streaming aggregation (Flink + sliding window), decision flow (rules + ML thresholds), feedback loop (labels → retrain → A/B)
5. **Wrap-up (5 min):** trade-offs (FP vs FN cost, explainability vs accuracy); failure modes (model timeout → rules-only); closing on the recall/precision tension

Key Talking Points

- “Layered defence — rules + ML + manual review” — every layer carries a different error type
- “Feature store is the core system” — fraud signal lives in aggregates over time; raw txn alone is useless
- “Streaming pipeline keeps the hot store fresh in < 1 s” — Flink on Kafka with sliding-window counters
- “Champion/challenger A/B with shadow bake” — model promotion never auto, always gated
- “Concept drift is what kills you, not bugs” — daily PSI/KS, weekly rolling AUC, ever-running shadow
- “Train/serve skew is the silent killer” — single feature definition library for both planes
- “Model timeout → fall back to rules-only” — never block the auth path on the model

Common Probe Questions & Answers

- **Q: Why not just rules?** A: They bleed against novel patterns; you'd need 10K+ rules within a year and they become unmaintainable. ML adapts to shifts that no analyst has written a rule for yet.
- **Q: Why not just ML?** A: Regulators require explanations for declines; rules are explainable by construction. And when a fraud ring suddenly hits, you need a hot-fix path that doesn't wait for a retrain.
- **Q: How do you handle class imbalance?** A: ~30 bps fraud → undersample non-fraud (10–50× ratio), use focal loss or class weights, optimise for AUC-PR not accuracy.

- **Q: What if the model server times out?** A: 30 ms hard timeout, fall back to rules-only with tighter thresholds, emit a metric, auto-revert the deploy after N timeouts in 5 min.
- **Q: How do you avoid train/serve skew?** A: Single feature library (e.g. Feast) emits both Spark batch and Flink streaming implementations from one source. PSI checks daily.
- **Q: How do you detect concept drift?** A: Daily PSI/KS on each feature, weekly rolling AUC on labelled traffic, an always-running shadow model, and customer-fraud-report spikes as a leading indicator.
- **Q: What about adversarial attacks?** A: Card-testing attacks pop the velocity rules; credential-stuffing pops the device aggregates; the rules engine is hot-fixable in minutes via config push so you can react faster than retraining.

KEY

Memorize These Numbers

1B txns/day · ~12K TPS sustained · ~50K TPS Black Friday peak · p99 inline < 50 ms · ~30 feature lookups / scored txn · ~1.6 TB hot Redis · ~30 bps fraud rate · nightly retrain on 90 d · 5% challenger A/B · 7 yr audit retention.

INSIGHT

Closing Line

“Fraud detection is about being ‘right enough, fast enough’ — never perfectly right, never instantaneous, but always deterministic, always explainable, and always a tiny bit ahead of the adversary. The architecture is rules + ML + humans, the operational spine is the feature store and the feedback loop, and the discipline is to keep the online and offline planes ruthlessly separate.”